On Node Ranking of Graphs

Yung-Ling Lai, Sheng-Hsiung Wang and Wei-Chun Lin
Department of Computer Science and Information Engineering
National Chiayi University, Chiayi, Taiwan
{ yllai, s0942783, s0942810}@mail.ncyu.edu.tw

Abstract

For a connected graph $G=(V,E)$, a node ranking labeling $C:V\to\{1,2,...k\}$ is a mapping such that for every path between any two vertices u, v with $C(u)=C(v)$, there exists at least one vertex w on the path with $C(w)>C(u)=C(v)$. The value $C(v)$ is called the rank of the vertex v and G is said to be k-rankable if there exists a node ranking labeling of G with maximum rank k. The node ranking problem is the problem of finding minimum k such that G is k-rankable. There are two versions of node ranking problem, namely offline and online. In off-line version, the node ranking problem deals with a graph where all vertices and edges are given in advance. In on-line version, the vertices are given one by one in an arbitrary order and only the edges of the induced subgraph of given vertices are known when the rank of each vertex has to be chosen. The rank can not be changed after it is assigned. This paper solved node ranking problem of corona graphs and mesh in off-line version, and gave on-line node ranking algorithm for sun graphs.

1 Introduction

Let $G=(V,E)$ be an undirected graph where V denotes the set of vertices and E denotes the set of edges. G is said to be *k-rankable* if there is a mapping $C:V\to\{1,2,...k\}$ such that every path between any two vertices u, v with $C(u)=C(v)$, there exists at least one vertex w on the path with $C(w)>C(u)=C(v)$. The mapping C is called a *node ranking labeling* of G and the value $C(v)$ is called the *rank* of the vertex v. The *node ranking number* $\chi_r(G)$ is the smallest integer k such that G is *k*-rankable. A node ranking labeling is an *optimal node ranking labeling* of G if its maximum rank is $\chi_r(G)$. The *node ranking problem* is the problem of finding the node ranking number $\chi_r(G)$ of a graph G. We said the node ranking problem is *solved* if a formula for the node ranking number is obtained or an algorithm of an optimal node ranking labeling is given. The node ranking problem is applicable in communication network design[3][15]; finding the minimum height of a node separator tree of a graph [3][8][15], which are extensively used in VLSI layout [11][14]; and developing algorithms for planning efficient assembly of products in manufacturing systems [4] etc. There are two versions of node ranking problem, off-line and on-line versions. In off-line version, the graph with all vertices and edges are given in advance. In on-line version, the vertices are given one by one in an arbitrary order together with the edges adjacent to the vertices that are already given. The rank has to be assigned in real time and once the rank is assigned to a vertex, it is not changeable. Similar to the offline version, the *on-line node ranking number* is denoted as $\chi_r^*(G)$, which is the smallest integer k such that G is on-line *k*-rankable.

Node ranking problem has been studied since 1980s. In the offline version, it is known that for a path P_n, $n\ge 1$, $\chi_r(P_n)=\lfloor\log_2 n\rfloor+1$ [5]. For a cycle C_n, $n\ge 3$, $\chi_r(C_n)=\lceil\log_2 n\rceil+1$ [1]. For wheel graphs $W_{1,n}$ with n vertices on the cycle, $n\ge 3$, $\chi_r(W_{n+1})=\lceil\log_2 n\rceil+2$ [7]; the node ranking number of the complete bipartite graph $K_{m,n}$ is $\chi_r(K_{m,n})=m+1$ for $m\le n$ [8]; and star graphs $S_n(=K_{1,n-1})$ with n vertices, $\chi_r(S_n)=2$ for $n\ge 3$ [13]. An upper bound of the node ranking number for an arbitrary tree with n nodes was given by [3], they proposed an $O(n\log_2 n)$ time optimal node ranking algorithm, which was further improved to $O(n)$ by [12]. There are fewer results in the on-line version. [1] gave tight bounds for paths and cycles as $\lfloor\log_2 n\rfloor+1<\chi_r^*(P_n)<2\lfloor\log_2 n\rfloor+1$ for $n\ge 1$; and $\lceil\log_2 n\rceil+1<\chi_r^*(C_n)<2\lceil\log_2 n\rceil-1$ for $n\ge 3$, respectively. The on-line version of a complete bipartite graph with $m\le n$ was proved to be $m+1\le\chi_r^*(K_{m,n})\le\min\{n+1,2m+1\}$ [8] and the on-line node ranking number of star graph is given by [13] as $\chi_r^*(S_n)=3$ for $n\ge 3$. There are several papers worked on on-line trees,

the best known on-line node ranking algorithm for trees which has $O(n^2)$ time complexity and $O(n)$ space complexity was given by [6]. [9] proposed an online ranking algorithm for trees using CREW PRAM model with $O(n^3/\log^2 n)$ processors. [10] provided an optimal online ranking algorithm for stars and an $O(n^3)$ time algorithm for arbitrary trees.

This paper established offline node ranking number for mesh and the corona of any two graphs and provided an on-line algorithm with tight bound of online node ranking number for sun graphs.

2 Offline Results for Corona Graphs

Given graphs G and H with n and m vertices respectively, the corona of G with respect to H denoted as $G \otimes H$, is the graph with vertex set $V(G \otimes H) = V(G) \cup \{n$ distinct copies of $V(H)$ denoted as $V(H_1), V(H_2), \cdots V(H_n)\}$ and the edge set $E(G \otimes H) = E(G) \cup \{n$ distinct copies of $E(H)$ denoted as $E(H_1), E(H_2), \cdots E(H_n)\} \cup \{(u_i, v) : u_i \in V(G), v \in V(H_i), 1 \le i \le n\}$. Figure 1 shows an example of graph $P_3 \otimes P_4$.

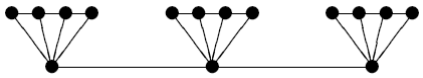

Figure1: An example of graph $P_3 \otimes P_4$.

Theorem 1: *Let $G \otimes H$ denote the corona of G with respect to H. Then the node ranking number $\chi_r(G \otimes H) = \chi_r(G) + \chi_r(H)$.*

Proof: First we show that $\chi_r(G \otimes H) \le \chi_r(G) + \chi_r(H)$. Let f_1 and f_2 be optimal node ranking labeling of G and H respectively. Let $V(G) = \{u_1, u_2, \cdots, u_n\}$ and $V(H) = \{v_1, v_2, \cdots, v_m\}$, $V(G \otimes H) = V(G) \cup \{v_{i,j} |$ which is the i-th copy of the j-th vertex of H, $1 \le i \le n, 1 \le j \le m\}$. Define f as $f(u_i) = f_1(u_i) + \chi_r(H)$ for $u_i \in V(G)$ and $f(v_{i,j}) = f_2(v_j)$ for $v_{i,j} \in V(H_i)$, then f is a node ranking labeling of $G \otimes H$ with maximum rank $\chi_r(G) + \chi_r(H)$, which implies $\chi_r(G \otimes H) \le \chi_r(G) + \chi_r(H)$.

Next we show that $\chi_r(G \otimes H) \ge \chi_r(G) + \chi_r(H)$. Suppose to the contrary, that $\chi_r(G \otimes H) < \chi_r(G) + \chi_r(H)$. Let g be an optimal node ranking of $G \odot H$.

Case1: $\forall 1 \le i \le n, 1 \le j \le m$, $g(u_i) > g(v_{i,j})$. Then $g(u_i) > \chi_r(H)$, $\forall u_i \in V(G)$. Define a labeling g' of G as $g'(u_i) = g(u_i) - \chi_r(H)$, then g' is a node ranking labeling of G with maximum rank $\chi_r(G \otimes H) - \chi_r(H) < \chi_r(G)$, which is a contradiction.

Case2: $g(u_i) < g(v_{i,j})$, for some $1 \le i \le n$, $1 \le j \le m$. Consider a labeling h of $G \otimes H$, which exchange the label of u_i with the minimum label of $v_{i,j}$ for those $v_{i,j}$ having label greater than u_i. That is, if y is the minimum label of $V(H_i)$ such that $g(u_i) < y$, then for all vertices $v_{i,j}$ with $g(v_{i,j}) = y$, let $h(v_{i,j}) = g(u_i)$, $h(u_i) = y$, and $h(x) = g(x)$ otherwise. Repeat the same process on h until $h(u_i) > h(v_{i,j})$ for all $1 \le i \le n, 1 \le j \le m$. Note that for each exchange, since it starts from the smallest label, the order of the ranks are not changed in graph H_i, and for $h(u_i) = h(u_k)$ we must have some j that $g(v_{i,j}) = g(u_k)$, since g is a node ranking labeling of graph $G \otimes H$, there must be a w on $u_i - u_k$ path with $h(w) > h(u_i) = h(u_k)$, which implies h is a node ranking labeling of $G \otimes H$. Since the maximum label of h is the same as the maximum label of g, which lead us back to case 1 and produce a contradiction. ■

Let $S_{n,m}$ be the graph which contains a n-cycle and each cycle vertex has m hairs. Since the graph has the shape of sun, we named it as "sun graph". Figure 2 shows an example of graph $S_{5,2}$.

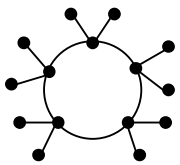

Figure 2: An example of graph $S_{5,2}$.

Since the sun graph $S_{n,m}$ may be viewed as $C_n \otimes \overline{K_m}$, following corollary comes directly from Theorem 1 by knowing that $\chi_r(C_n) = \lceil \log_2 n \rceil + 1$ (from [1]) and $\chi_r(\overline{K_m}) = 1$.

Corollary 1: *Let* $S_{n,m}$ *be the sun graph of order* $n(m+1)$. *Then the node ranking number* $\chi_r(S_{n,m}) = \lceil \log_2 n \rceil + 2$.

3 Offline Results for Mesh Graphs

In this section, we proposed an off-line ranking algorithm for mesh graphs $P_n \times P_n$. The algorithm produced a node ranking labeling of $P_n \times P_n$ for odd n and $n \geq 15$, for even n and $n \geq 18$. Figure 3 shows an example of the graph $P_6 \times P_6$.

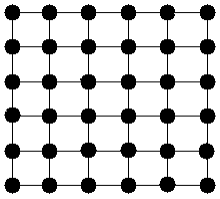

Figure 3: An example of graph $P_6 \times P_6$.

Algorithm Offline_Ranking_Mesh
Input : $n, n \geq 15$ if n is odd and $n \geq 18$ if n is even. Each vertex is denoted as $v_{i,j}$ for $1 \leq i, j \leq n$.
Output : An off-line rank assignment of $P_n \times P_n$.
Method:
$array[y][x]$ holds the label of vertex $v_{y,x}$
If (n is odd) **Then**
Step1: **If** $x + y = even$ **Then** $array[y][x] = 1$.
Step2: **For** $x = 1$ **to** $\lceil n/2 \rceil$ **do**
Dividing the graph by $x + y = n$.
For $x = \lceil n/2 \rceil$ **to** n **do**
Dividing the graph by $x + y = n + 2$.
Step3: **For** $x = 2$ **to** $\lfloor n/2 \rfloor$ **do**
Dividing the graph by $x - y = 1$.
For $x = \lceil n/2 \rceil + 1$ **to** $n - 1$ **do**
Dividing the graph by $y - x = 1$.
// The graph is then divided into four isomorphic subgraphs $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ as shown in Figure 4.

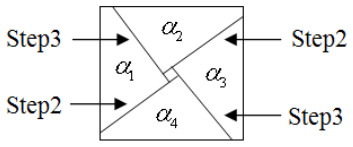

Figure 4: An example of graphs $\alpha_1, \alpha_2, \alpha_3, \alpha_4$.

Step4: In the α_1 graph, $i = \lfloor n/2 \rfloor - 1$; call *Alpha* (i); use the label for corresponding vertices in α_2 to α_4.
// i means the number of nodes on the edge of subgraph α_1.
Else If (n is even and $n \equiv 0 \bmod 4$) **Then**
Step1: **If** $x + y = even$ **Then** $array[y][x] = 1$.
Step2: **For** $x = n/4 + 3$ **to** $n/2 + 4$ **do**
Dividing the graph by $x + y = n/2 + 5$.
For $x = n/4 + 3$ **to** $n/2 + 1$ **do**
Dividing the graph by $x - y = 1$.
For $x = n/2$ **to** $n/2 + 1$ **do**
Dividing the graph by $x + y = n + 1$.
For $x = n/2$ **to** $3n/4 - 2$ **do**
Dividing the graph by $y - x = 1$.
For $x = n/2 - 3$ **to** $3n/4 - 2$ **do**
Dividing the graph by $x + y = 3n/2 - 3 = u_0$.
// The graph is divided into two isomorphic subgraphs ω_1, ω_2 as shown in Figure 5.

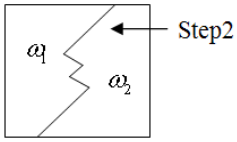

Figure 5: An example of graphs ω_1, ω_2.

Step3: In the ω_1 graph, Dividing the graph by $x + y = n/2 + 5 = u_1$. (Repeat the same processes for the corresponding vertices in ω_2.)
// The graph is divided into two subgraphs $\varepsilon_1, \varepsilon_2$ as shown in Figure 6.

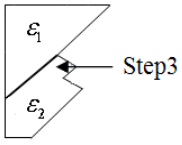

Figure 6: An example of graphs $\varepsilon_1, \varepsilon_2$.

Step4: Dividing ε_1 and ε_2 by $x - y = 1$ .and $y - x = n/2 - 1 = v_0$ respectively.
// The ε_1 graph is divided into two subgraphs α_1, α_2 and the ε_2 graph is divided into two subgraphs δ_1, δ_2 as shown in Figure 7.

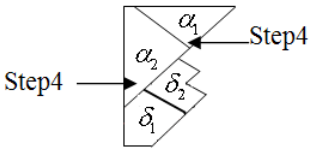

Figure 7: An example of graphs $\alpha_1, \alpha_2, \delta_1, \delta_2$.

Step5: In the α_1 graph, $i = n/4$; call *Alpha* (i);
In the α_2 graph, $j = \lfloor (3n - 12)/8 \rfloor$; call *Alpha* (j);
In δ_1 and δ_2, there are lines of vertices denoted as $e_1, e_2, \cdots, e_l$, such that

$x+y=u_1+2c, 1\le c\le (u_0-u_1)/2-1$ where $l=3n/4-4-\lfloor (3n-12)/8\rfloor$. Let $j=\lfloor l/2\rfloor$.

Dividing δ_1 and δ_2 by e_j and e_{j+1} respectively.

Let $u_2=x+y$ for the vertices $v_{y,x}$ on line e_j and $u_3=x+y$ for the vertices $v_{y,x}$ on line e_{j+1}.

// The δ_1 graph is divided into two subgraphs δ_{11},δ_{12} and the δ_2 graph is divided into two subgraphs δ_{21},δ_{22} as shown in Figure 8.

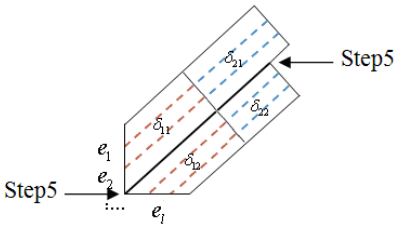

Figure 8: An example of graphs $\delta_{11},\delta_{12},\delta_{21},\delta_{22}$.

Step6: In δ_{11}, Dividing the graph by $y-x=n/2+2\lfloor n/2\rfloor-1=v_1$.

// Graph δ_{11} is divided into two subgraphs α_{11},β_{11} as shown in Figure 9.

In δ_{12}, Dividing the graph by $y-x=n/2+2\lceil (n-4)/16\rceil+1=v_2$.

// Graph δ_{12} is divided into two subgraphs α_{12},β_{12} as shown in Figure 9.

In δ_{21}, $p=(u_3-u_1)/2-1$; $q=(v_0-1)/2$; call *Beta* (p,q);

In δ_{22}, $p=(u_0-u_3)/2-1$; $q=(v_0-3)/2$; call *Beta* (p,q);

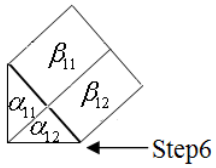

Figure 9: An example of graphs $\alpha_{11},\beta_{11},\alpha_{12},\beta_{12}$.

Step7: In α_{11}, $i=(u_0-v_1)/2-2$; call *Alpha* (i);

In β_{11}, let $p=(u_2-u_1)/2-1$; $q=(v_1-v_0)/2-1$; call *Beta* (p,q);

In α_{12}, $i=(u_0-u_2)/2-2$; call *Alpha* (i);

In β_{12}, let $p=(u_0-u_2)/2-1$; $q=(v_2-v_0)/2-1$; call *Beta* (p,q);

Else // n is even and $n\not\equiv 0 \bmod 4$.

Step1: **If** $x+y=even$ **Then** $array[y][x]=1$.

Step2: **For** $x=\lfloor n/4\rfloor+3$ **to** $n/2+3$ **do**

Dividing the graph by $x+y=n/2+4$.

For $x=\lfloor n/4\rfloor+3$ **to** $n/2+1$ **do**

Dividing the graph by $x-y=1$.

For $x=n/2$ **to** $n/2+1$ **do**

Dividing the graph by $x+y=n+1$.

For $x=n/2$ **to** $3\lfloor n/4\rfloor$ **do**

Dividing the graph by $y-x=1$.

For $x=n/2-2$ **to** $3\lfloor n/4\rfloor$ **do**

Dividing the graph by $x+y=3n/2-2=u_0$.

// The graph is divided into two isomorphic subgraphs ω_1,ω_2 which is the same as in Figure 5.

Step3: In the ω_1 graph, Dividing the graph by $x+y=n/2+2\lceil (n-2)/8\rceil=u_1$. (Repeat the same processes for the corresponding vertices in ω_2.)

// The graph is divided into two subgraphs $\varepsilon_1,\varepsilon_2$ as shown in Figure 6.

Step4: Dividing ε_1 and ε_2 by $x-y=1$ and $y-x=n/2=v_0$ respectively.

// The graph ε_1 is divided into two subgraphs α_1,α_2 and the graph ε_2 is divided into two subgraphs δ_1,δ_2 as shown in Figure 7.

Step5: In α_1, $i=n/4$; call *Alpha* (i);

In α_2, $i=\lfloor (3\lfloor n/4\rfloor-1)/2\rfloor$; call *Alpha* (i);

In δ_1 and δ_2, there are lines of vertices $e_1,e_2,\cdots,e_l$ such that $x+y=u_1+2c$, where $1\le c\le (u_0-u_1)/2-1, l=\lceil (n+2)/8\rceil+(n+2)/4-3$.

Let $j=\lfloor l/2\rfloor$, dividing δ_1 and δ_2 by and e_{j+1} respectively.

Let $u_2=x+y$ for the vertices $v_{y,x}$ on line e_j and $u_3=x+y$ for the vertices $v_{y,x}$ on line e_{j+1}.

// Graphs δ_1 and δ_2 are divided into subgraphs δ_{11},δ_{12} and δ_{21},δ_{22} respectively, as shown in Figure 8.

Step6: Dividing the graph δ_{11} by $y-x=n/2+2\lceil (n+2)/8\rceil-2=v_1$.

// Graph δ_{11} is divided into two subgraphs α_{11},β_{11} as shown in Figure 9.

Dividing graph δ_{12} by $y-x=n/2+2\lceil (n+2)/16\rceil=v_2$.

// Graph δ_{12} is divided into two subgraphs α_{12},β_{12} as shown in Figure 9.

In graph δ_{21}, let $p=(u_3-u_1)/2-1$; $q=(v_0-1)/2$; call *Beta* (p,q);

In graph δ_{22}, let $p=(u_0-u_3)/2-1$; $q=(v_0-3)/2$; call *Beta* (p,q);

Step7: In graph α_{11}, let $i=(u_0-v_1)/2-2$; call *Alpha* (i);

In graph β_{11}, let $p=(u_2-u_1)/2-1$; $q=(v_1-v_0)/2-1$; call *Beta* (p,q);

In α_{12}, let $i=(u_0-u_2)/2-2$; call *Alpha* (i);

In graph β_{12}, let $p=(u_0-u_2)/2-1$; $q=(v_2-v_0)/2-1$; call *Beta* (p,q);

End of Algorithm Offline_Ranking_Mesh_Graph

Procedure Alpha (int i **)**

If $(i\ge 3)$ **Then** {

$p=\lfloor i/3\rfloor$; $q=i-p-1$;

// p means the number of nodes of the dividing line P.

$p_1 = p + p_1$;

Dividing the graph by $I: y - x = 2p_1 + 2c - 1$.

Dividing the graph by $P: x + y = 2p_1 + 2c + 1$.

// The graph α_1 is divided into three subgraphs $\alpha_{11}, \alpha_{12}, \alpha_{13}$ as shown in Figure 10.

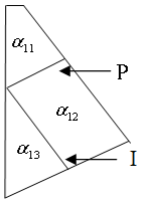

Figure 10: Subgraphs $\alpha_{11}, \alpha_{12}, \alpha_{13}$.

In graph α_{11}, call *Alpha* (p);

If $\left(\left(\chi_r(\alpha_{11}) \le \chi_r(P_p) + p\right)$ and $\left(\chi_r(\alpha_{11}) \le 2\chi_r(P_p) - 1\right)$ and $\left(\chi_r(\alpha_{11}) \ge P_p\right)\right)$ **Then** {

In the α_{12} graph, dividing the graph by $a_0: x + y = n - 4$.

$q = q - 2$;

}

Else If $\left(\left(\chi_r(\alpha_{11}) \le \chi_r(P_p) + p\right)$ and $\left(\chi_r(\alpha_{11}) > 2\chi_r(P_p) - 1\right)$ and $\left(\chi_r(\alpha_{11}) \ge P_p\right)\right)$ **Then** {

Dividing graph α_{12} by $a_0: x + y = n - 6$.

$q = q - 3$;

}

Else {

Dividing graph α_{12} by $a_0: x + y = n - 8$.

$q = q - 4$;

}

// The graph α_{12} is divided into two subgraphs β_1, β_2 as shown in Figure 11.

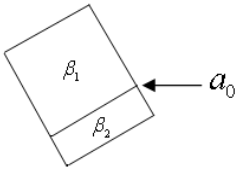

Figure 11: An example of graphs β_1, β_2.

In β_1, call *Beta* (p, q);

$i = i - 1 - p$; c++;

In β_2, call *Beta* (p, q);

In α_{13}, call *Alpha* (i);

Return $\chi_r(\alpha_1)$; // ranking of α_1

}

Else If $(i = 2)$ **Then** Rank the vertices individually with 2, 3, 4.

Else Rank the vertex with 2. // $i = 1$

End of Procedure Alpha

Procedure Beta (int p**, int** q **)**

If $(q \ge 3)$ **Then** {

In β_1, there are dividing lines : $a_1, a_2, \cdots, a_q$ as shown in Figure 12.

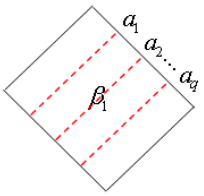

Figure 12: An example of dividing lines $a_1, a_2, \cdots, a_q$.

If q is odd **Then** {

Dividing the graph by $a_{\lceil q/2 \rceil}$.

// The dividing line divides the graph into two the same graphs.

$q = \lfloor q/2 \rfloor$;

}

If q is even **Then** {

Dividing the graph by $a_{q/2+1}$.

$q = q/2$;

}

}

Else If $(q = 2)$ **Then** call *Gamma* (p);

Else Rank by optimal ranking as path // $q = 1$ call *Beta* (p, q);

End of Procedure Beta

Procedure Gamma (int p **)**

If $(p \ge 3)$ **Then** {

// graph β_1 is divided into two subgraphs: γ_1, γ_2 by a_1.

In γ_1, there are dividing lines: $b_1, b_2, \cdots, b_p$.

Figure 13 shows γ_1, γ_2 and $b_1, b_2, \cdots, b_p$.

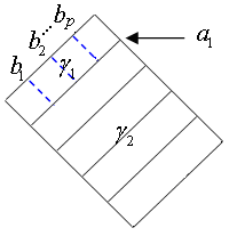

Figure 13: Graphs γ_1, γ_2 and $b_1, b_2, \cdots, b_p$.

If p is odd **Then** {

Dividing the graph by $b_{\lceil p/2 \rceil}$.

// The graph is divided into two identical subgraphs.

$p = \lfloor p/2 \rfloor$;

}

If p is even **Then** {

Dividing the graph by $b_{p/2+1}$.

$p = p/2$;

}

}

Else If $(p = 2)$ **Then** Rank the nodes using 2 to 5.
Else Rank the nodes individually with 2 and 3.
call *Gamma* (p);
End of Procedure Gamma

Theorem 2: *The labeling given by Algorithm Offline_Ranking_Mesh is a node ranking labeling of* $P_n \times P_n$ *for odd* n *and* $n \geq 15$, *for even* n *and* $n \geq 18$.

Proof: In the algorithm, we always assign greater rank for the dividing vertices, and on every path between two nodes of the same rank there is at least one dividing line, so the labels given by the algorithm must follow the node ranking condition for $P_n \times P_n$. ■

Theorem 3: *The maximum rank given by Algorithm Offline_Ranking_Mesh for odd* n *and* $n \geq 15$ *is* $n + \lfloor n/2 \rfloor + \lceil (n+1)/6 \rceil + \lfloor (n+1)/6 \rfloor + 8\lceil (n-1)/12 \rceil - 10 + x$ *; and for even n,* $n \equiv 0 \bmod 4$ *and* $n \geq 20$ *is* $n + n/4 + \lceil (n-2)/8 \rceil - 1 + \max\{3(n/4 - 2 + \lfloor n/8 \rfloor), n/2 + 4\lceil n/8 \rceil - 5\}$ *; and for even n* $n \not\equiv 0 \bmod 4$ *and* $n \geq 18$ *is* $n + \lceil n/4 \rceil + \lceil (n-2)/8 \rceil - 1 + \max\{3(\lceil n/4 \rceil - 2 + \lfloor n/8 \rfloor), 2\lfloor n/4 \rfloor + \lceil (n+3)/12 \rceil + \lfloor (n+3)/12 \rfloor + 8\lceil (n-3)/24 \rceil - 9 + x\}$, *where*

$$x = \begin{cases} 0 & \text{if } y = 0, 2 \\ 2 & \text{if } y = 4 \\ 4 & \text{if } y = 6, 8 \\ 5 & \text{if } y = 10 \end{cases} \quad \text{for} \quad y = n - \left(3 + 12\lfloor (n-3)/12 \rfloor\right).$$

Theorem 3 gave us an upper bound for the node ranking number of a mesh $P_n \times P_n$ for $n \geq 17$. For $n \leq 16$, Table 1 shows the node ranking number of $P_n \times P_n$ which is obtained by a brute force computer testing, and is denoted by the formula that represents the relation between n and $\chi_r(P_n \times P_n)$.

Table 1: $\chi_r(P_n \times P_n)$ for $n \leq 16$.

n	$\chi_r(P_n \times P_n)$
$1 \leq n \leq 7$	$2n-1$
$8 \leq n \leq 10$	$2n$
11	$2n+1$
12	$2n$
13	$2n+2$
14	$2n+1$
15	$2n+3$
16	$2n+2$

4 Online Results

In this section, we proposed an on-line ranking algorithm for sun graphs $S_{n,m} = C_n \otimes \overline{K_m}$, and analyze both time and space complexities of our algorithm. Notice that there are $n(m+1)$ vertices in $S_{n,m}$. Let v_1 to $v_{n(m+1)}$ represent the order of the coming vertices.

Algorithm Online_Ranking_Sun_Graph
We use structure array Sun[] to record the information of each node, and use value_record[] as the counter of the index value's appearance.
Input:
n: the number of vertices on the cycle of $S_{n,m}$.
m: the number of vertices on the hairs of $S_{n,m}$.
v_i for $1 \leq i \leq n(m+1)$ the vertices together with the adjacent vertices v_j for $1 \leq j < i$.
Output:
An online node ranking labeling of $S_{n,m}$.
Method:
For $i = 1$ **to** $(m+1)n$ **do {**
If v_i has no adjacent vertices, $rank(v_i) = 2$
Else if v_i has only one neighbor v_j for $j < i$ and v_j has two neighbors with degree >1 then $rank(v_i) = 1$
Else {

1. Set the occurrence record to zero except rank 1 (which is set to 2)
2. Applying online tree algorithm given in [5] to loop through each adjacent path, go as long as possible, to find the max value j that has occurred at least twice
3. Find minimum available rank k that is greater than j
4. Set $rank(v_i) = k$

}
}
End of Algorithm Online_Ranking_Sun_Graph

Theorem 4: *Algorithm Online_Ranking_Sun has* $O((mn)^3)$ *time complexity and* $O(mn)$ *space complexity.*

Proof:
First we analyze the time complexity of the algorithm. Since a Sun graph with $(m+1)n$ vertices has exactly $(m+1)n$ edges, each time when insert a new node v_i, the looping for finding max j used online tree algorithm which takes $O(i^2)$ times , so after $(m+1)n$ vertices inserted, the whole process will be at most $1^2 + 2^2 + \cdots + ((m+1)n)^2 = O((mn)^3)$ times. For the space complexity, we use structure array with $(m+1)n$ elements to store the node plus one array to record the rank occurrence, the total space complexity then is $O(mn) + O(mn) = O(mn)$. ■

Theorem 5: *For a sun graph* $S_{n,m}$ *with* $n \geq 7$ *is* $\lceil \log_2 n \rceil + 2 \leq \chi_r^*(S_{n,m}) \leq \lceil \log_2 n \rceil + 6m$.

Proof:

Since the online node ranking number is at least as much as the offline node ranking number, we know that $\lceil \log_2 n \rceil + 2 \leq \chi_r^*(S_{n,m})$. Let $S_{n,m}$ be the graph which contains a *n*-cycle v_i for $1 \leq i \leq n$ and each cycle vertex has *m* hairs denoted as $u_{i,j}$ for $1 \leq i \leq n, 1 \leq j \leq m$. If we use adversary analysis, for the best we can do, saving rank 1 for the vertices which are known as hair, assume that the vertices of $S_{n,m}$ are coming as follows:

- v_1, v_2, v_3, v_4 coming in first, so they get labels 2,3,2,4.
- For $3 \leq i \leq n-3$, all $u_{i,j}$ for $1 \leq j \leq m$ comes before v_{i+2}.
- Then comes $u_{n-2,j}$ for $1 \leq j \leq m$ and $u_{i,j}$ for $i = 1, 2, n-1$ and $1 \leq j \leq m$.
- At last, v_n then $u_{n,j}$ for $1 \leq j \leq m$.

In this coming order, before v_n comes in, no vertex can be recognized as end vertex, hence online tree algorithm worked for this case which is the worst case and our algorithm guarantee the maximum rank is no more than $\lceil \log_2 n \rceil + 6m$, hence we have $\chi_r^*(S_{n,m}) \leq \lceil \log_2 n \rceil + 6m$, which complete the proof.■

5 Conclusion

There are several papers discussed the node ranking problem. In this paper, we presented off-line node ranking number for the corona graphs and gave labeling algorithm for mesh. We also gave on-line algorithm for sun graphs. Both time and space complexities of our algorithms are analyzed.

References

[1] E. Bruoth and M. Hornak, "On-line ranking number for cycles and paths," *Discussiones Mathematicae, Graph Theory* 19, pp. 175-197, 1999.

[2] J.S. Deogun, T. Kloks, D. Kratsch, and H. Muller, "On vertex ranking for permutation and other graphs," *Lecture Notes in Computer Science* 775, pp. 747-758, 1994.

[3] A. V. Iyer, "Optimal node ranking of trees," *Information Processing Letters* 28, pp. 225-229, 1988.

[4] Md. A. Kashem, X. Zhou, T. Nishizeki, "Algorithm for generalized vertex-rankings of partial k-trees," *Theoretical Computer Science 240*, pp. 407-427, 2000.

[5] M. Katchalski, W. McCuaig, and S. Seager, "Ordered colorings," *Discrete Mathematics* 142, pp. 141-154, 1995.

[6] Y.-L. Lai and Y.-M. Chen, "An Improves On-line Node Ranking Algorithm of Trees," *Proceedings of the 23rd Workshop on Combinatorial Mathematics and Computation Theory*, pp. 345-348, April 19-20, 2006.

[7] Y.-L. Lai and Y.-M. Chen, "On Node Ranking of Graphs under Strong Orientation," *Proceedings of the 24th Workshop on Combinatorial Mathematics and Computation Theory*, April 27-28, 2007.

[8] Y.-L. Lai and Y.-M. Chen, "On Node Ranking of Complete r-partite Graph," *Journal of Combinatorial Mathematics and Combinatorial Computing* 67, 2008.

[9] C. Lee and J. S. Juan, "Parallel Algorithm for On-Line Ranking in Trees," *Proc. Of the 22nd Workshop on Combinatorial Mathematics and Computational Theory*, May 20-21, pp. 151-156, 2005.

[10] C. Lee and J. S. Juan, Jun. 2005, "On-Line Ranking Algorithms for Trees," *Proc. of Int. Conf. on Foundations of Computer Science*, Monte Carlo Resort, Las Vegas, Nevada, USA,. June 27-30, pp. 46-51, 2005.

[11] C.E. Leiserson, "Area-efficient graph layouts (for VLSI)," *Proceedings of the 21st Annual IEEE Symposium on Foundations of Computer Science*, pp. 270-281, 1980.

[12] A. A. Schaffer, "Optimal node ranking of trees in linear time," *Information Processing Letters* 33, pp. 91-96, 1989.

[13] G. Semanisin and R. Sotak, Kosice, "A note on on-line ranking number of graphs," *Czechoslovak Mathematical Journal* 56, pp. 591-599, 2006.

[14] A. Sen, "On a graph partition problem with application to VLSI layout," *Information Processing Letters* 43, pp. 87-94, 1992.

[15] P. de la Torre, R. Greenlaw, A. A. Schäffer, "Optimal edge ranking of trees in polynomial time," *Algorithmca* 13, pp. 592-618, 1995.